

Phase 2: Web Application Development & EC2 Deployment

Challenge: Build a functional frontend application, host it on the EC2 web server, configure load balancing, offload static assets to S3, and set up Auto Scaling for high availability.

Host Web App on EC2 Instance

Activity	Status
Develop the Student Study Planner frontend (HTML, CSS, JavaScript)	Done
Connect to the EC2 instance via SSH	Done
Install and enable Nginx on Amazon Linux 2023	Done
Upload application files to the instance via SCP	Done
Deploy files to <code>/var/www/study-planner</code> and set Nginx ownership	Done
Configure Nginx to serve the app and expose a health check endpoint	Done
Verify the application is reachable over HTTP in a browser	Done

A simple web application called **Student Study Planner** was developed using HTML, CSS, and JavaScript. The application allows users to manage study tasks and monitor progress through an intuitive interface.

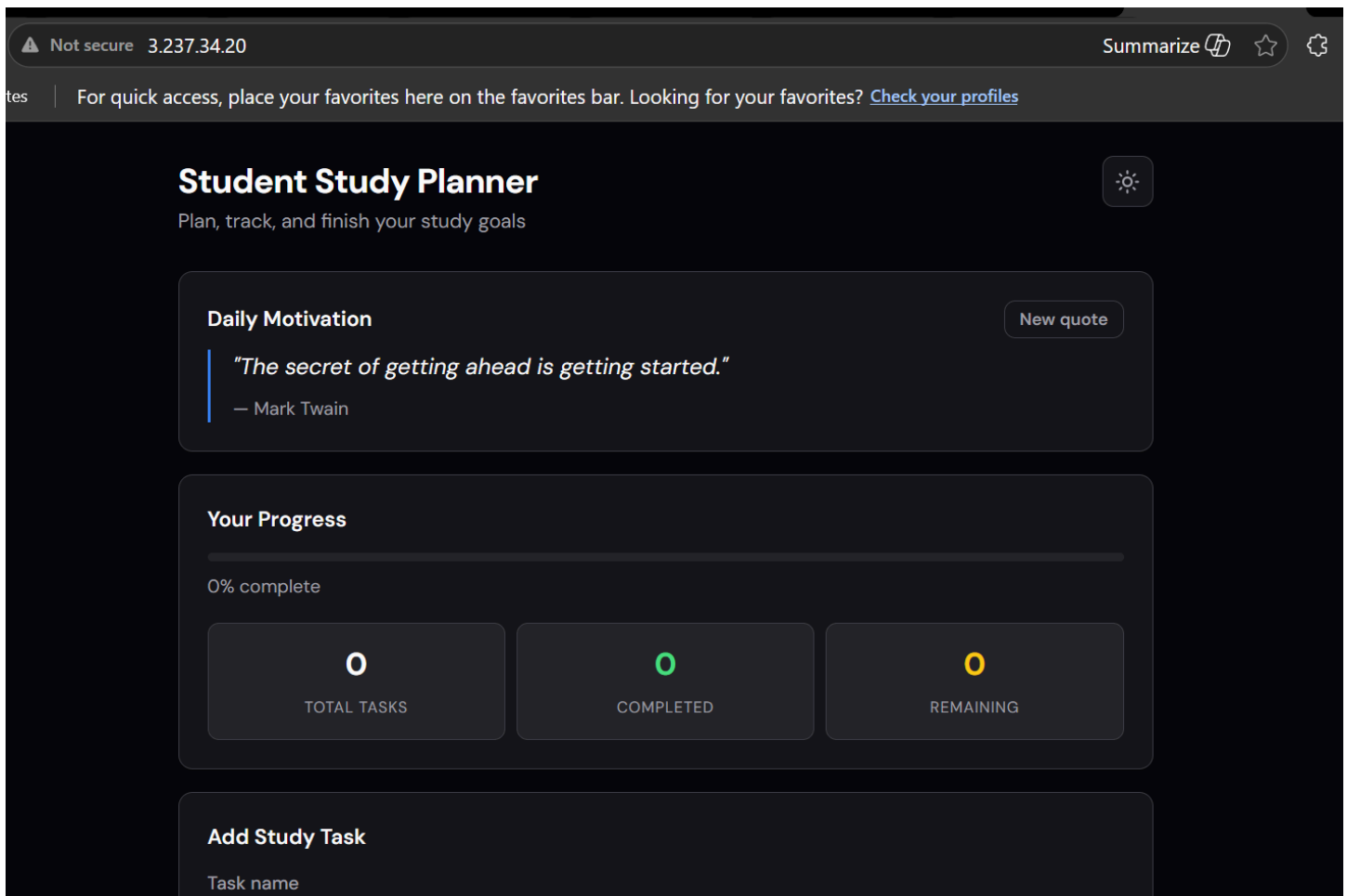
Web Application

The app lives in the `app/` directory and includes:

File	Purpose
<code>index.html</code>	Main UI — task form, progress tracker, daily motivation
<code>css/styles.css</code>	Layout, theming (light/dark), responsive styles
<code>js/app.js</code>	Task CRUD, filtering, localStorage persistence
<code>js/config.js</code>	S3 logo URL and app configuration

health.html

Plain-text health check response for load balancer probes



SSH Connection to EC2

Connected to the Capstone-WebServer instance using the team SSH key pair:

```
ssh -i eddie-key.pem ec2-user@3.237.34.20
```

```
PS C:\Users\eddie\OneDrive\Desktop\aws-capstone-project> ssh -i eddie-key.pem ec2-user@3.237.34.20
The authenticity of host '3.237.34.20 (3.237.34.20)' can't be established.
ED25519 key fingerprint is SHA256:RPZc8RS4QZFUGr6wIRKIixXU8cOYAniE6nJ1FWVsniQ.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])?
yes
Warning: Permanently added '3.237.34.20' (ED25519) to the list of known hosts.
#_
~\#####\
~\###|
~\#/ ____ https://aws.amazon.com/linux/amazon-linux-2023
~\~' '->
~\
~\
~\
~\m/'
[ec2-user@ip-172-31-71-213 ~]$
```

Install Nginx

Nginx was installed, enabled at boot, and started on Amazon Linux 2023:

```
sudo dnf install -y nginx sudo systemctl enable nginx sudo systemctl start nginx
```

```
Installing      : nginx-filesystem-1:1.30.2-1.amzn2023.0.1.noarch      1/7
Installing      : nginx-mimetypes-2.1.49-3.amzn2023.0.3.noarch        2/7
Installing      : libunwind-1.4.0-5.amzn2023.0.3.x86_64              3/7
Installing      : gperftools-libs-2.9.1-1.amzn2023.0.3.x86_64        4/7
Installing      : nginx-core-1:1.30.2-1.amzn2023.0.1.x86_64          5/7
Installing      : generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch    6/7
Installing      : nginx-1:1.30.2-1.amzn2023.0.1.x86_64              7/7
Running scriptlet: nginx-1:1.30.2-1.amzn2023.0.1.x86_64              7/7
Verifying       : generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch    1/7
Verifying       : gperftools-libs-2.9.1-1.amzn2023.0.3.x86_64        2/7
Verifying       : libunwind-1.4.0-5.amzn2023.0.3.x86_64              3/7
Verifying       : nginx-1:1.30.2-1.amzn2023.0.1.x86_64              4/7
Verifying       : nginx-core-1:1.30.2-1.amzn2023.0.1.x86_64          5/7
Verifying       : nginx-filesystem-1:1.30.2-1.amzn2023.0.1.noarch      6/7
Verifying       : nginx-mimetypes-2.1.49-3.amzn2023.0.3.noarch        7/7

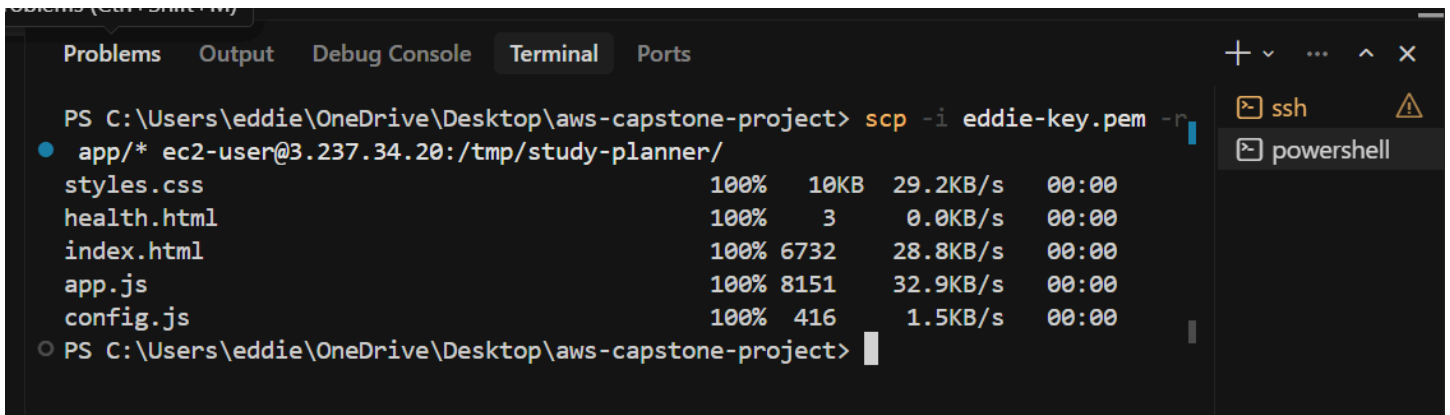
Installed:
  generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch
  gperftools-libs-2.9.1-1.amzn2023.0.3.x86_64
  libunwind-1.4.0-5.amzn2023.0.3.x86_64
  nginx-1:1.30.2-1.amzn2023.0.1.x86_64
  nginx-core-1:1.30.2-1.amzn2023.0.1.x86_64
  nginx-filesystem-1:1.30.2-1.amzn2023.0.1.noarch
  nginx-mimetypes-2.1.49-3.amzn2023.0.3.noarch

Complete!
Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service → /usr/lib/systemd/system/nginx.service.
[ec2-user@ip-172-31-71-213 ~]$
```

Upload & Deploy Application Files

Application source files were copied from the local workstation to the instance using SCP, then moved into the Nginx document root:

```
scp -i eddie-key.pem -r app/* ec2-user@3.237.34.20:/tmp/study-planner/
sudo mkdir -p /var/www/study-planner sudo cp -r /tmp/study-planner/*
/var/www/study-planner/ sudo chown -R nginx:nginx /var/www/study-planner
```



```
PS C:\Users\eddie\OneDrive\Desktop\aws-capstone-project> scp -i eddie-key.pem -r
app/* ec2-user@3.237.34.20:/tmp/study-planner/
styles.css                                100%  10KB  29.2KB/s  00:00
health.html                             100%    3    0.0KB/s  00:00
index.html                             100% 6732  28.8KB/s  00:00
app.js                                 100% 8151  32.9KB/s  00:00
config.js                              100%  416   1.5KB/s  00:00
PS C:\Users\eddie\OneDrive\Desktop\aws-capstone-project>
```

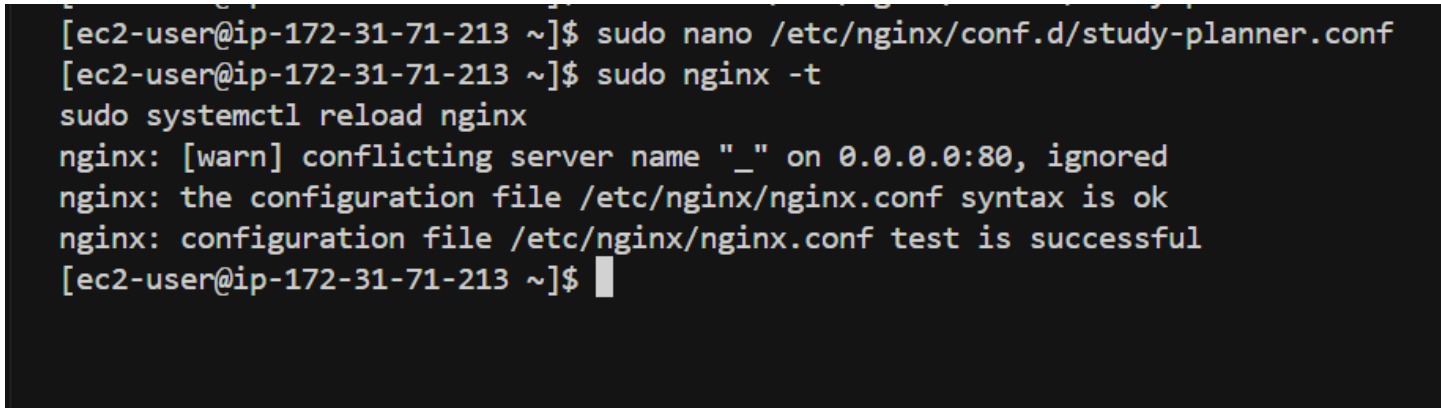
Nginx Site Configuration

A site block was created at /etc/nginx/conf.d/study-planner.conf:

```
server {
    listen 80;
    server_name _;
    root /var/www/study-planner;
    index index.html;
    location / {
        try_files $uri $uri/ /index.html;
    }
    location = /health.html {
        access_log off;
        return 200 "OK\n";
        add_header Content-Type text/plain;
    }
}
```

The configuration was validated and Nginx was reloaded:

```
sudo nginx -t
sudo systemctl reload nginx
```



```
[ec2-user@ip-172-31-71-213 ~]$ sudo nano /etc/nginx/conf.d/study-planner.conf
[ec2-user@ip-172-31-71-213 ~]$ sudo nginx -t
sudo systemctl reload nginx
nginx: [warn] conflicting server name "_" on 0.0.0.0:80, ignored
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
[ec2-user@ip-172-31-71-213 ~]$
```

Live Verification

The application is live at **http://3.237.34.20** and serves the Student Study Planner UI over HTTP (port 80).

Property	Value
Web root	/var/www/study-planner
Nginx config	/etc/nginx/conf.d/study-planner.conf
Public URL	http://3.237.34.20
Health check	http://3.237.34.20/health.html

Task 2 — Configure Target Group for EC2

Activity	Status
Create target group (<code>capstone-web-tg</code>) with HTTP on port 80	Done
Configure health check path (<code>/health.html</code>)	Done
Review and create the target group in the AWS Console	Done
Register <code>Capstone-WebServer</code> EC2 instance as a target	Done

An **Application Load Balancer Target Group** was created to route traffic to the EC2 web server, reusing the `/health.html` endpoint for health checks.

Target Group Settings

Property	Value
Name	<code>capstone-web-tg</code>
Target type	Instance
Protocol : Port	HTTP : 80
Protocol version	HTTP1
VPC	<code>vpc-00a64ca8744d9cb02</code> (default)
IP address type	IPv4
Health check path	<code>/health.html</code>
ARN	<code>arn:aws:elasticloadbalancing:us-east-1:610356897914:targetgroup/capstone-web-tg/fbd644598296d872</code>

capstone-web-tg

Actions

Details

arn:aws:elasticloadbalancing:us-east-1:610356897914:targetgroup/capstone-web-tg/fbd644598296d872

Target type

Instance

IP address type

IPv4

Protocol : Port

HTTP: 80

Protocol version

HTTP1

VPC

vpc-00a64ca8744d9cb02

Load balancer

None associated

0

Total targets

0

Healthy

0

Unhealthy

0

Unused

0

Initial

0

Draining

0

Anomalous

Targets

Monitoring

Health checks

Attributes

Tags

Registered targets (0)

Anomaly mitigation: Not applicable

Deregister

Register targets

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

Filter targets

1

Instance ID

Name

Port

Zone

Health status

Health status details

Administrative o...

Override details

Launch time

No registered targets

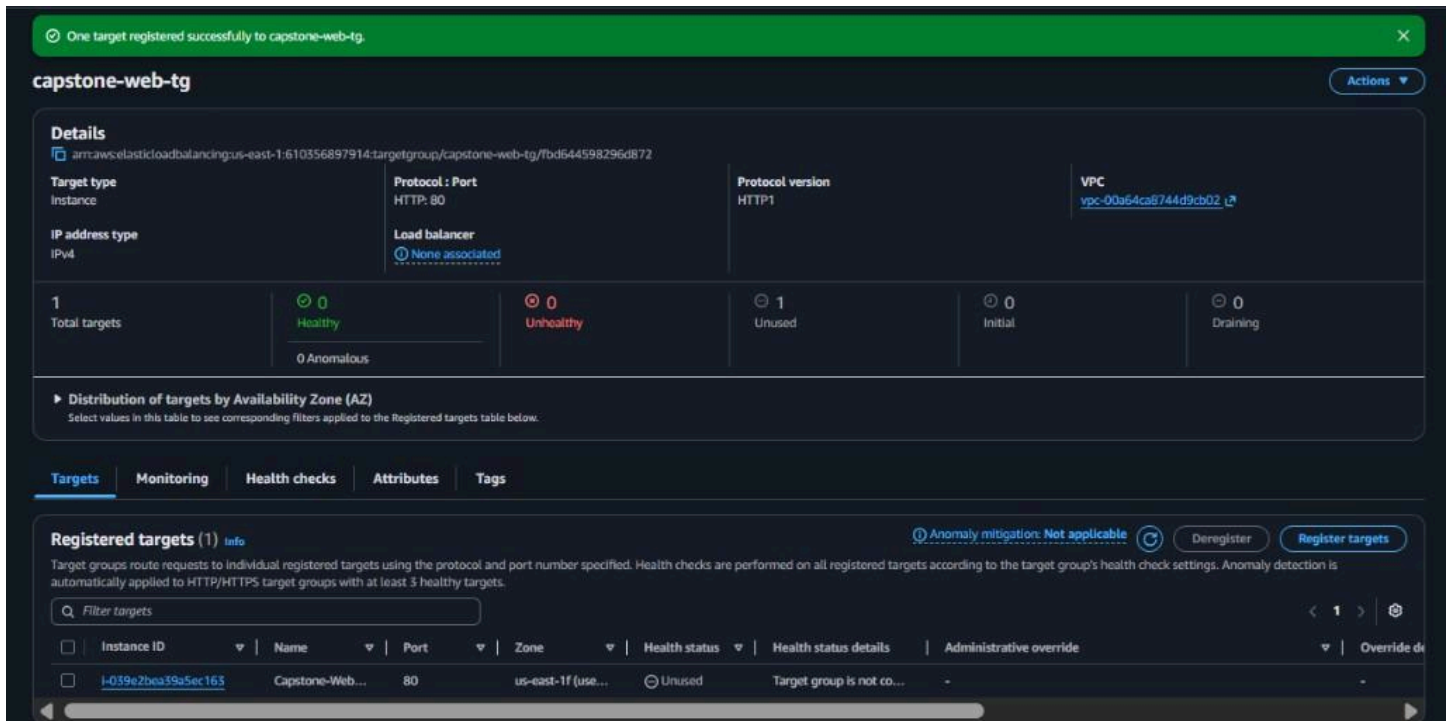
You have not registered targets to this group yet

Register targets

Register EC2 Instance

Capstone-WebServer was registered as a target on port 80:

Property	Value
Instance ID	i-039e2bea39a5ec163
Name	Capstone-WebServer
Port	80
Availability Zone	us-east-1f



Note: The target health status initially showed **Unused** because no load balancer was attached. Once the ALB was associated, health checks started and the instance updated to **Healthy**.

Task 3 — Configure Application Load Balancer (ALB)

Activity	Status
Create internet-facing Application Load Balancer (<code>study-planner-alb</code>)	Done
Configure listener on Port 80 (HTTP)	Done
Attach default forwarding action to target group <code>capstone-web-tg</code>	Done
Verify ALB DNS resolves and routes traffic to the EC2 instances	Done
Test health checks and verify "Healthy" status in target group	Done

An **Application Load Balancer (ALB)** was deployed to distribute incoming traffic across backend instances in multiple Availability Zones.

Load Balancer Configuration

Property	Value
----------	-------

Name	study-planner-alb
Scheme	Internet-facing
IP address type	IPv4
Listener	HTTP : 80
Action	Forward to capstone-web-tg
DNS Name	study-planner-alb-1113325153.us-east-1.elb.amazonaws.com
VPC	vpc-00a64ca8744d9cb02

The screenshot displays the 'study-planner-alb' details page in the AWS console. The 'Details' tab is active, showing the following information:

- Load balancer type:** Application
- Scheme:** Internet-facing
- Status:** Active (indicated by a green checkmark)
- Hosted zone:** Z35SXDOTRQ7X7K
- VPC:** vpc-00a64ca8744d9cb02
- Availability Zones:** subnet-0619586d1700d0ad7 (us-east-1b (use1-az2)), subnet-09dca073c71774379 (us-east-1a (use1-az1))
- Load balancer IP address type:** IPv4
- Date created:** June 10, 2026, 20:11 (UTC+00:00)
- Load balancer ARN:** arn:aws:elasticloadbalancing:us-east-1:610356897914:loadbalancer/app/study-planner-alb/d58f27a8fce000ab
- DNS name info:** study-planner-alb-1113325153.us-east-1.elb.amazonaws.com (A Record)

Navigation tabs at the bottom include: Listeners and rules, Network mapping, Resource map, Security, Monitoring, Integrations, Attributes, Capacity, and Tags.

Live Verification of Target Health

After the ALB was created, active health probes verified Nginx and all instances showed **Healthy** status. Accessing the ALB DNS name (study-planner-alb-1113325153.us-east-1.elb.amazonaws.com) in a browser confirmed the Student Study Planner loaded successfully through the load balancer.

The screenshot displays the 'capstone-web-tg' details page in the AWS console. The 'Details' tab is active, showing the following information:

- Target type:** Instance
- Protocol : Port:** HTTP: 80
- Protocol version:** HTTP1
- VPC:** vpc-00a64ca8744d9cb02
- IP address type:** IPv4
- Load balancer:** study-planner-alb

Summary statistics:

- 4** Total targets
- 4** Healthy (0 Anomalous)
- 0** Unhealthy
- 0** Unused
- 0** Initial
- 0** Draining

A section titled 'Distribution of targets by Availability Zone (AZ)' is visible at the bottom, with a note: 'Select values in this table to see corresponding filters applied to the Registered targets table below.'

Task 4 — Offload Static Assets to S3

Activity	Status
Sync application code and static assets to S3 bucket	Done

Configure public access and CORS rules on S3 bucket	Done
Update application asset configuration in <code>app/js/config.js</code>	Done

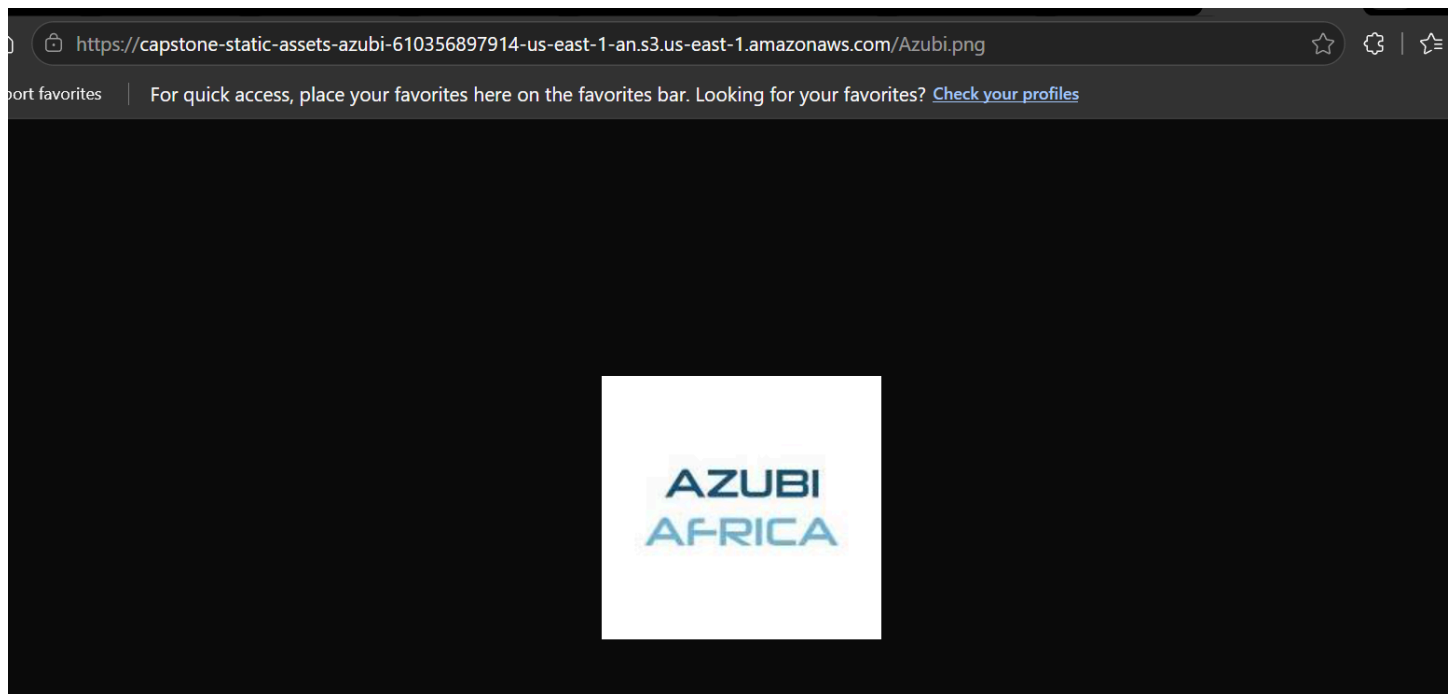
The static assets of the **Student Study Planner** are synchronized to a public S3 bucket and served directly to improve performance.

Synchronization Command

```
aws s3 sync app/ s3://capstone-static-assets-azubi-610356897914-us-east-1-an/
--profile cloud-project
```

S3 Bucket Settings

Property	Value
Bucket Name	<code>capstone-static-assets-azubi-610356897914-us-east-1-an</code>
Region	<code>us-east-1</code>
Static Asset Base URL	<code>https://capstone-static-assets-azubi-610356897914-us-east-1-an.s3.us-east-1.amazonaws.com</code>



Task 5 — Configure Auto Scaling Group (ASG)

Activity	Status
----------	--------

Create custom AMI (<code>Capstone-WebServer-AMI</code>) from active EC2 instance	Done
Create Launch Template (<code>capstone-web-lt</code>) using the custom AMI	Done
Configure Auto Scaling Group (<code>capstone-web-asg</code>)	Done
Link ASG to <code>capstone-web-tg</code> to automate target registration	Done

An Auto Scaling Group was configured to scale instances dynamically between 1 and 3 web servers for high availability.

Configuration Summary

Property	Value
AMI Name	<code>Capstone-WebServer-AMI</code> (<code>ami-011769ff5f024dd25</code>)
Launch Template	<code>capstone-web-lt</code> (<code>lt-017bd876a4f79d816</code>)
Instance Type	<code>t3.micro</code> (Free-Tier eligible)
ASG Name	<code>capstone-web-asg</code>
Desired / Min / Max	1 / 1 / 3
Availability Zones	All 6 default subnets in <code>us-east-1</code>
Health Check Type	ELB (Grace Period: 300 seconds)

capstone-web-asg

capstone-web-asg Capacity overview

arn:aws:autoscaling:us-east-1:610356897914:autoScalingGroup:56beaa94-4df0-4cba-858c-8b45311f3ff2:autoScalingGroupName/capstone-web-asg

Desired capacity

1

Scaling limits

1 - 3

Desired capacity type

Units (number of instances)

Status

At desired capacity

Date created

Wed Jun 10 2026 22:34:57 GMT+0000 (Greenwich Mean Time)

Details

Integrations

Automatic scaling

Instance management

Instance refresh

Activity

Monitoring

Tags

Launch template

Launch template

lt-017bd876a4f79d816

capstone-web-lt

Version

1

Description

v1

AMI ID

ami-011769ff5f024dd25

Security groups

-

Storage (volumes)

-

Instance type

t3.micro

Security group IDs

sg-04e6b787ff7ea5574

Key pair name

eddie-key

Owner

arn:aws:iam::610356897914:user/Edmund

Create time

Wed Jun 10 2026 22:33:08 GMT+0000 (Greenwich Mean Time)

Request Spot Instances

No

Once active, the ASG launched a clone instance (i-0a0bf40efd7d380bc) and automatically registered it into the Load Balancer target group. The ASG activity history confirmed the scale-out event, and the EC2 dashboard showed both the original and cloned instances in a running state with healthy target group registration.

AWS Certificate Manager (ACM) — Custom Domain & HTTPS

Property	Value
Domain Name	capstonestudyplanner.duckdns.org
Certificate Type	Public Certificate
Validation Method	DNS Validation
Region	us-east-1 (N. Virginia)
Status	Issued

An SSL/TLS certificate was requested through ACM for the custom domain to enable HTTPS in a future phase (CloudFront + ALB HTTPS listener).

Request public certificate

Domain names
Provide one or more domain names for your certificate.

Fully qualified domain name [Info](#)

capstonestudyplanner.duckdns.org

[Add another name to this certificate](#)

You can add additional names to this certificate. For example, if you're requesting a certificate for "www.example.com", you might want to add the name "example.com" so that customers can reach your site by either name.

Allow export [Info](#)

☒ **Disable export**
Use this certificate only with integrated AWS services. The private key for this certificate will be disallowed for exporting from AWS.

☐ **Enable export**
Export this certificate and private key for use with any TLS workflow. ACM will charge your account based on the requested domains when the certificate is issued for the first time and for each renewal.

Validation method [Info](#)

Select a method for validating domain ownership.

☒ **DNS validation - recommended**
Choose this option if you are authorized to modify the DNS configuration for the domains in your certificate request.

Certificates > feff9074-0ef8-4669-9980-f02c02e5b6a7

Successfully requested certificate with ID feff9074-0ef8-4669-9980-f02c02e5b6a7
A certificate request with a status of pending validation has been created. Further action is needed to complete the validation and approval of the certificate. [View certificate](#)

feff9074-0ef8-4669-9980-f02c02e5b6a7 [Delete](#)

Certificate status

Identifier	Status
feff9074-0ef8-4669-9980-f02c02e5b6a7	☒ Pending validation Info

ARN
arn:aws:acm:us-east-1:610356897914:certificate/feff9074-0ef8-4669-9980-f02c02e5b6a7

Type
Amazon Issued

Next Steps

Phase 2 (Tasks 1–5) is complete. Upcoming work includes:

- Configure CloudFront CDN distribution in front of the ALB
- Enable HTTPS with the ACM certificate on the CloudFront distribution and ALB listener
- Configure custom domain routing via DuckDNS → CloudFront
- Automate infrastructure with IaC — CloudFormation or Terraform (future phases)

Last updated: June 10, 2026 — Phase 2 Tasks 1–5 documented (EC2 hosting, ALB, S3 static assets, Auto Scaling Group, ACM certificate).